CPE 314 — Software Design

University of Cebu

Computer Engineering Department

Pet Shelter & Adoption Tracker

Project Design Document

Group Name / No.:	***Group 3***
Members:	
	Shann Michael Agang - UI Designer / Developer
	Sofia Yvonne Enovero - Team Lead / Developer
	Kyle Cheznie Taghoy - Quality Assurance
	Aeisha Mae Elizar - Database Admin
	Cleint Lausa - Developer
Instructor:	Richyll Son
Term:	First Semester, S.Y. 2026-2027
Document Version / Date:	*V 0.3 - 11/13/2026*

Table of Contents

Pet Shelter & Adoption Tracker....1
Table of Contents....2
Revision History....3
1. Introduction & Project Overview....4
2. Preliminary ER Sketch & Team Role....4
2.2 Preliminary entities and key attributes....5
2.3 Preliminary relationships and cardinality....5
2.4 Team members and primary roles....5
3. System Architecture & Object-Oriented Design....6
3.1 Class Hierarchy - PawTrack: Pet Shelter & Adoption Tracker....6
3.2 Single Responsibility Principle Justification Table....6
3.3 Design Rationale: PawTrack....8
4. Data Model & Database Design....9
4.1 ER Diagram....9
4.2 SQLite schema....9
5. API Integration Design....10
5.1 API 1: Dog CEO API (https://dog.ceo/dog-api/)....10
5.2 API 2: TheCatAPI (https://thecatapi.com/)....11
5.3 Error Handling....11
5.4 Example JSON Responses....11
6. Concurrency Design....12
6.1 Background Operations....12
6.2 How the Background Work Runs....13
6.3 Keeping the UI Responsive....13
7. Data Mining & Visualization Design....13
7.1 Data Mining and Aggregation....13
7.2 Recursive Classification Analysis....14
7.3 Data Visualization....14

Revision History

Version	Date	Author	Changes
0.1	Nov. 6, 2026	Sofia Yvonne Enovero	Add section 1 One paragraph Pitch
		Shann Michael Agang	Add Section 2 Entity-Relationship
		Kyle Cheznie Taghoy	Update ER Sketch, add a risk question Update erd.png
		Aeisha Mae Elizar	Update Section 2 Table 2, Update risks questions
		Cleint Lausa	Add Task Split Table, Update risk questions
0.2	Oct. 27, 2026	Kyle Cheznie Taghoy	Add Class Hierarchy Diagram
		Cleint Lausa	Update SRP Table
		Aeisha Mae Elizar	Add Design Rationale
		Sofia Yvonne Enovero	Update ER diagram
		Shann Michael Agang	Add Database SQL Schema
0.3	Nov. 3, 2026	Kyle Cheznie Taghoy	Add the subsection 5.1 and 5.2 tables
		Aeisha Elizar	Add Error handling and json example
		Shann Michael Agang	Add Concurrency design section
		Sofia Yvonne Enovero	Add 7.1 table and 7.2 subsection
		Cleint Lausa	Add Data Visualization

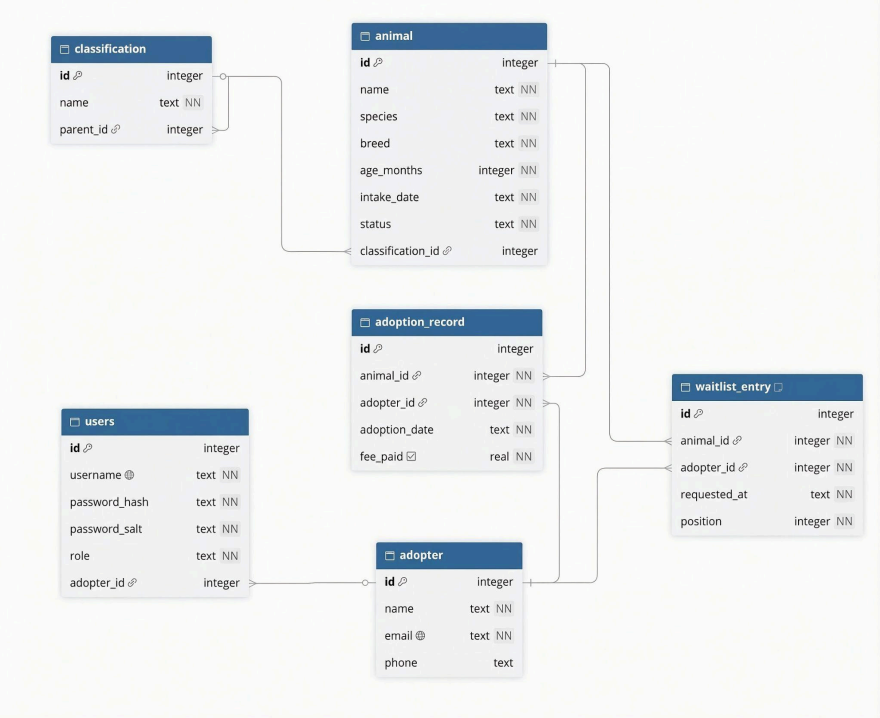
1. Introduction & Project Overview

The Pet Shelter & Adoption Tracker (PawTrack) is a desktop application designed to help animal shelters manage animal records, adoption activities, adopter information, and adoption waitlists. It provides shelter staff with an organized platform for maintaining animal information, recording adopter details, tracking adoption progress, and monitoring changes in animal status.

The system aims to improve the efficiency and organization of shelter operations through features such as animal record management, adoption waitlist handling, adoption tracking, CSV data import, and analytics. It also incorporates external pet image APIs to retrieve sample animal images for dogs and cats. Object-oriented programming principles and appropriate data structures are applied to support the system's functionality and data management.

PawTrack is developed using Java, Java Swing, SQLite, and JDBC. It is intended for shelter-side use, with ADMIN and USER roles providing access to the application's features based on their assigned permissions. Adopters, including prospective adopters, are maintained as records within the system but do not log in or access the application directly. Core operations, including database management, waitlist processing, CSV importing, analytics, and status history tracking, are performed locally using the SQLite database. An internet connection is required only for retrieving external pet images through the integrated APIs.

2. Preliminary ER Sketch & Team Role


classification
id integer
name text NN
parent_id integer
animal
id integer
name text NN
species text NN
breed text NN
age_months integer NN
intake_date text NN
status text NN
classification_id integer
adoption_record
id integer
animal_id integer NN
adopter_id integer NN
adoption_date text NN
fee_paid real NN
waitlist_entry
id integer
animal_id integer NN
adopter_id integer NN
requested_at text NN
position integer NN
users
id integer
username text NN
password_hash text NN
password_salt text NN
role text NN
adopter_id integer
adopter
id integer
name text NN
email text NN
phone text

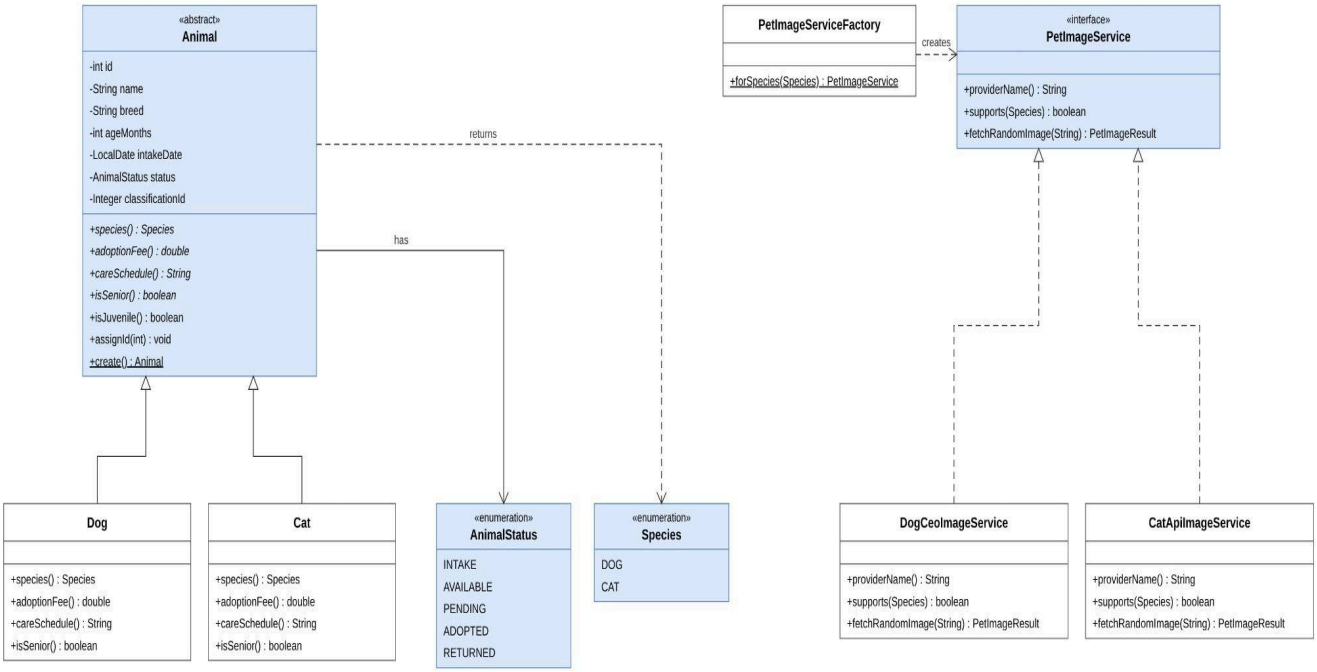
2.2 Preliminary entities and key attributes		
Entity	**Key Attributes**	**Description**
Classification	id (PK), name, parent_id (FK → Classification.id, nullable)	Stores animal classifications and supports hierarchical parent-child relationships.
Animal	id (PK), name, species, breed, age_months, intake_date, status, classification_id (FK → Classification.id)	Stores animal information, shelter intake details, current status, and classification.
Adopter	id (PK), name, email (unique), phone	Stores information about individuals interested in adopting animals.
AdoptionRecord	id (PK), animal_id (FK → Animal.id), adopter_id (FK → Adopter.id), adoption_date, fee_paid	Records completed adoption transactions and their associated animal and adopter.
WaitlistEntry	id (PK), animal_id (FK → Animal.id), adopter_id (FK → Adopter.id), requested_at, position	Records adoption interest and maintains the order of prospective adopters for an animal.

2.3 Preliminary relationships and cardinality				
Parent Entity	**Cardinality**	**Child Entity**	**Foreign Key**	**Description**
Classification	1:M	Classification	parent_id (nullable)	Self-referencing taxonomy tree of any depth, e.g. Dog > Sporting Group > Retriever > Golden Retriever
Classification	1:M	Animal	classification_id	Each animal may belong to one classification.
Animal	1:M	AdoptionRecord	animal_id	An adopter can have multiple adoption records.
Adopter	1:M	AdoptionRecord	adopter_id	An adopter can adopt multiple animals
Animal	1:M	WaitlistEntry	animal_id	An animal can have many adopters waiting
Adopter	1:M	WaitlistEntry	adopter_id	An adopter can have multiple waitlist entries for different animals.

2.4 Team members and primary roles	
Member	**Primary Responsibility**
Sofia Yvonne Enovero	Team Lead / Developer
Shann Michael Agang	UI Designer/Developer
Cleint Lausa	Developer
Kyle Cheznie Taghoy	QA (Quality Assurance)
Aeisha Mae Elizar	Database Admin

3. System Architecture & Object-Oriented Design

3.1 Class Hierarchy - PawTrack: Pet Shelter & Adoption Tracker



3.2 Single Responsibility Principle Justification Table

Class hierarchy (animals)		
Class	**Single responsibility**	**Why it follows SRP**
Animal (abstract)	Holds the data every animal shares (id, name, breed, age, intake date, status, classification) and validates it. It declares the species-specific methods species(), adoptionFee(), careSchedule(), and isSenior().	It contains no fee rules, database code, GUI code, or API code. It changes only when shared animal data or the common contract changes.
Dog	Applies the dog rules: puppy, standard, and senior fees, the senior age of 84 months, and the dog care schedule.	It changes only when dog-specific fee or care rules change.
Cat	Applies the cat rules: kitten, standard, and senior fees, the senior age of 120 months, and the cat care schedule.	It changes only when cat-specific fee, senior age threshold, or care schedule rules change.
Species (enum)	Defines the supported species (DOG, CAT) and their display label.	It changes only when a species is added.
AnimalStatus (enum)	Defines the animal's status values (INTAKE, AVAILABLE, PENDING, ADOPTED, RETURNED) and converts them to and from labels.	It changes only when the animal status lifecycle or its labels change.

Image API classes		
Class	**Single responsibility**	**Why it follows SRP**
PetImageService (interface)	Defines what the app needs from an image source: providerName(), supports(Species), and fetchRandomImage(String).	The GUI depends on a common interface rather than a specific API provider.
DogCeoImageService	Communicates with the Dog CEO API and parses its JSON response.	It changes only when the Dog CEO API integration changes.
CatApiImageService	Communicates with TheCatAPI, handles its optional API key, and parses its JSON response.	It changes only when TheCatAPI integration changes.
PetImageServiceFactory	Selects the appropriate PetImageService implementation based on the animal species.	Provider selection logic is kept separate from the GUI and individual API services.
PetImageResult	Holds the result of an image request, including the image URL, species, breed, and provider.	It represents image-result data without handling API requests or GUI operations.
HttpJson	Performs HTTP GET requests and returns the response as JSON text.	Shared network-request logic is centralized instead of being repeated in individual API services.

Services and Data Structures		
Class	**Single responsibility**	**Why it follows SRP**
WaitlistService	Manages each animal's first-come, first-served queue of interested adopters.	It handles waitlist operations separately from database and GUI responsibilities.
StatusHistoryService	Manages the stack of animal status changes to support undo operations.	It focuses on status history and stack operations.
StatusChange	Holds one status change, including the animal and its previous and new statuses.	It represents status-change data without managing the undo process itself.
ClassificationService	Manages classification search, recursive traversal, and path-building operations.	It centralizes classification-related operations separately from the data structure itself.
ClassificationNode	Represents one node in the classification tree and maintains its child nodes.	It focuses on storing the structure of an individual classification node without handling search operations.
ShelterSeeder	Populates an empty database with sample shelter records.	Sample-data initialization is kept separate from regular application operations.
CsvImportService	Processes CSV records and imports the corresponding animal, adopter, and adoption information into the database.	CSV parsing, validation, and import operations are separated from the GUI.

Database layer		
Class	**Single responsibility**	**Why it follows SRP**
Database	Opens the SQLite connection, enables foreign-key enforcement, and initializes the database schema.	Database connection and schema initialization are centralized.

Database layer		
AnimalDao, AdopterDao, AdoptionRecordDao, ClassificationDao, WaitlistDao	Each DAO performs database operations for its corresponding table using SQL and prepared statements.	Database operations are separated by entity, reducing dependencies between table operations.
Adopter, AdoptionRecord	Hold adopter and adoption-record information.	They represent application data without directly handling database queries or interface operations.

GUI layer		
Class	**Single responsibility**	**Why it follows SRP**
MainFrame	Builds and coordinates the main application window and its interface components.	It manages the main window while delegating business operations to services and other application components.
AnimalTableModel	Supplies animal data to the Swing table.	It handles the presentation of animal records in the table.
ImageWorker	Loads and processes sample pet images in a background thread.	Image loading is separated from the main interface thread to help keep the application responsive.
ImportWorker	Runs CSV import operations in a background thread and reports progress to the interface.	Long-running import operations are kept separate from the Swing Event Dispatch Thread.

Security Layer		
Class	**Single Responsibility**	**Why It Follows SRP**
Credentials	Holds the username and password information submitted during login.	Login input data is kept separate from authentication and permission logic.
LoginManager	Coordinates the authentication process and determines whether submitted credentials are valid.	Authentication coordination is centralized instead of being handled directly by the GUI.
PasswordHasher	Handles password hashing and password verification using the selected secure hashing method.	Password-processing logic is isolated from the login interface and account management.
AccessPolicy	Determines whether an authenticated user has permission to perform a particular operation based on their role.	Authorization rules are centralized rather than scattered throughout the application.

3.3 Design Rationale: PawTrack

PawTrack uses object-oriented programming, design patterns, and data structures to keep the system organized, reusable, and maintainable. Its components are separated according to their responsibilities, with DAO classes handling database operations, services managing business logic, and interfaces separating external API functions. Queues, stacks, and trees support the waitlist, undo feature, and animal classification. Background processing keeps the interface responsive, while the planned security components will provide separate access levels for staff and adopters.

4. Data Model & Database Design

4.1 ER Diagram

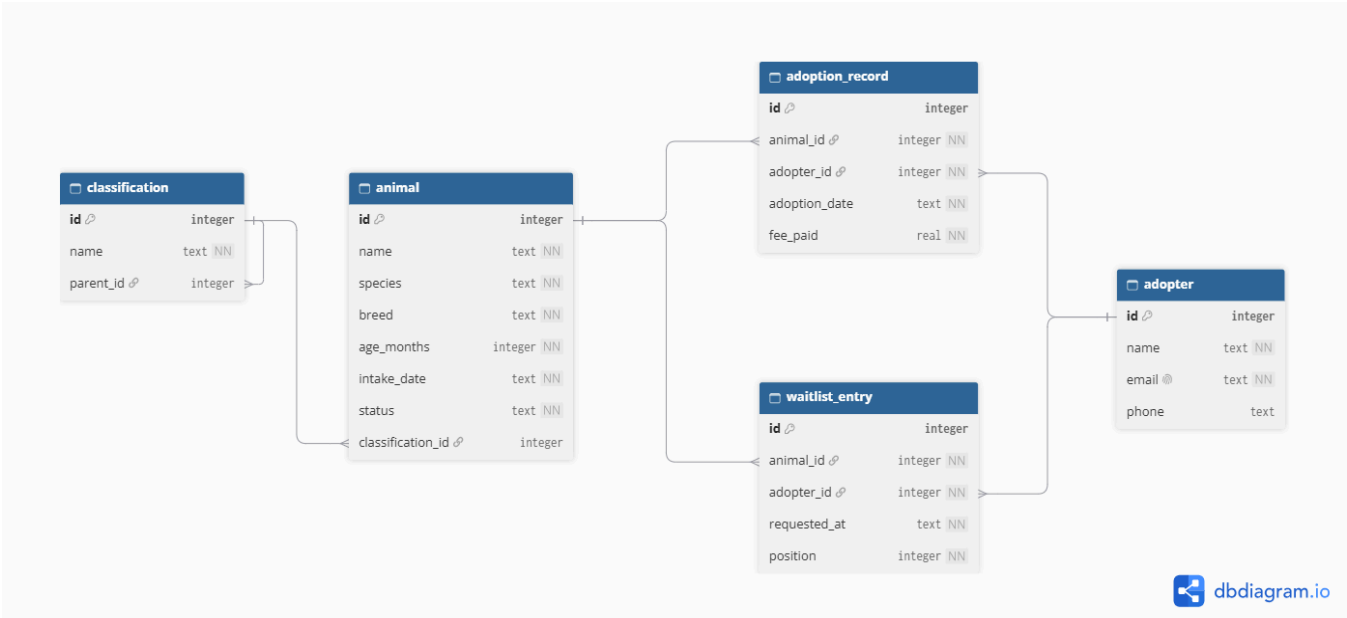


Figure 4.1 Finalized ERD. PK = primary key, FK = foreign key, NN = NOT NULL

This section supersedes the preliminary ER sketch in Section 2.

4.2 SQLite schema

```
-- Database file: pawtrack.db (created automatically by Database.java on first run)
-- The app runs PRAGMA foreign_keys = ON on every connection.

CREATE TABLE IF NOT EXISTS classification (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  parent_id INTEGER REFERENCES classification(id) ON DELETE CASCADE
);
```

```
CREATE TABLE IF NOT EXISTS adopter (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  email TEXT NOT NULL COLLATE NOCASE UNIQUE,
  phone TEXT
);

CREATE TABLE IF NOT EXISTS animal (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  species TEXT NOT NULL,
  breed TEXT NOT NULL,
  age_months INTEGER NOT NULL CHECK (age_months >= 0),
  intake_date TEXT NOT NULL,
  status TEXT NOT NULL,
  classification_id INTEGER REFERENCES classification(id)
);
CREATE TABLE IF NOT EXISTS adoption_record (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  animal_id INTEGER NOT NULL REFERENCES animal(id) ON DELETE CASCADE,
  adopter_id INTEGER NOT NULL REFERENCES adopter(id) ON DELETE CASCADE,
  adoption_date TEXT NOT NULL,
  fee_paid REAL NOT NULL DEFAULT 0
);

CREATE TABLE IF NOT EXISTS waitlist_entry (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
  animal_id INTEGER NOT NULL REFERENCES animal(id) ON DELETE CASCADE,
  adopter_id INTEGER NOT NULL REFERENCES adopter(id) ON DELETE CASCADE,
  requested_at TEXT NOT NULL,
  position INTEGER NOT NULL,
  UNIQUE(animal_id, adopter_id)
);
```

5. API Integration Design

PawTrack uses two public REST APIs to retrieve sample pet images: Dog CEO for dogs and TheCatAPI for cats. Both integrations follow a common PetImageService interface, while PetImageServiceFactory selects the appropriate service based on the animal's species. Each service sends HTTP GET requests through HttpJson, parses the JSON response using org.json, and returns a PetImageResult containing the image URL and related information. The application displays the retrieved image in the Sample Pet Image panel.

5.1 API 1: Dog CEO API (https://dog.ceo/dog-api/)

The Dog CEO API provides dog images and a catalogue of breeds. It needs no API key.

Table 4. Dog CEO API Integration

Endpoint (HTTP GET)	JSON Response Shape	Parsing and Use in PawTrack

Table 4. Dog CEO API Integration		
https://dog.ceo/api/breeds/list/all	message: an object that maps each breed to a list of sub-breeds. status: request status.	The app reads the object and converts breed and sub-breed pairs into readable names, for example retriever + golden becomes “Golden Retriever”. It uses this list to find the API path that matches the animal’s recorded breed.
https://dog.ceo/api/breed/{path}/images/random	message: a string containing one image URL. status: request status.	The app extracts the image URL and stores it in a PetImageResult. For example, the path retriever/golden returns a Golden Retriever image.
https://dog.ceo/api/breeds/image/random	message: a string containing a random dog image URL. status: request status.	The app uses this when the animal’s breed is blank, is not in the breed list, or the breed list cannot be loaded, so a dog image is still shown.

5.2 API 2: TheCatAPI (https://thecatapi.com/)

TheCatAPI provides cat images with breed information. It supports API-key authentication but also works without a key at a low rate limit. When configured, the key is read from the CAT_API_KEY environment variable or the cat.api.key system property and sent in the x-api-key header.

Table 5. TheCatAPI Integration		
Endpoint (HTTP GET)	**JSON Response Shape**	**Parsing and Use in PawTrack**
https://api.thecatapi.com/v1/images/search?limit=1&has_breeds=1	A JSON array with one object containing the image id, url, width, height, and a breeds array.	The app parses the array with org.json, extracts the image url, and stores it in a PetImageResult. The image is a random cat image, not matched to the animal’s breed. The caption below the image shows the provider and the animal’s own recorded breed.

5.3 Error Handling

PawTrack keeps working when an external API is unavailable. The image requests run in the background (see Section 6), and the GUI handles these cases:

- Slow or unreachable service: connection and read timeouts stop the request from waiting forever.
- Unsuccessful HTTP response: a status outside 200 to 299 is reported as an error that includes the status code.
- Invalid or missing JSON data: a response that cannot be parsed, or that has no image URL, is reported as an error.
- Image not available: the panel shows “Image unavailable”, the status bar says the fetch failed, and the problem is written to the Activity Log.
- Missing breed: the Dog CEO service falls back to a general random dog image.

A failed request never stops the rest of the application. Recording interest, marking animals adopted, importing history, filtering, undoing status changes, and viewing analytics all use the local SQLite database and do not depend on either API.

5.4 Example JSON Responses

- **Dog CEO API, /breeds/list/all: each breed maps to a list of its sub-breeds.**

```
{
  "message": {
    "labrador": [],
    "retriever": ["chesapeake", "curly", "flatcoated", "golden"]
  },
  "status": "success"
}
```

- **Dog CEO API, /breed/{path}/images/random and /breeds/image/random: both return one image URL in message.**

```
{
  "message": "https://images.dog.ceo/breeds/retriever-golden/n02099601_1024.jpg",
  "status": "success"
}
```

- **TheCatAPI, /images/search?limit=1&has_breeds=1: an array with one image object.**

```
[
  {
    "id": "aXYvRd7Do",
    "url": "https://cdn2.thecatapi.com/images/aXYvRd7Do.jpg",
    "width": 1445,
    "height": 1080,
    "breeds": [
      { "id": "abys", "name": "Abyssinian" }
    ]
  }
]
```

6. Concurrency Design

PawTrack uses Java Swing's SwingWorker to run long operations on background threads, so the Event Dispatch Thread (EDT), which handles clicks and repainting, never blocks.

6.1 Background Operations

Table 6. Background Operations and Threading Approach		
Operation	**Background Worker**	**Threading Approach**
Bulk CSV Import	ImportWorker	SwingWorker<CsvImportResult, Void>. doInBackground() calls CsvImportService.importFile(), which reads and validates each row and saves the animals, adopters, and adoption records to SQLite. It reports progress with setProgress().
Pet Image Retrieval	ImageWorker	SwingWorker<ImageWorker.Result, Void>. doInBackground() asks the PetImageService for an image URL, downloads the image bytes, and scales the image. It returns the image and its provider information.

6.2 How the Background Work Runs

The import is the longest operation. ImportWorker runs it in doInBackground() and publishes a percentage (0 to 100) through the standard progress property. The image worker makes two network calls, one for the image URL and one for the image itself. Neither runs on the EDT.

6.3 Keeping the UI Responsive

Swing components may only be changed on the EDT. SwingWorker delivers its progress and state change events on the EDT, so the GUI listens for them:

- While the import runs: the progress event moves the progress bar and its text, and the status bar says the import is running.
- When a worker is finished: the state changes to DONE and the GUI calls get() to read the result. For an import, it shows the summary in the status bar and Activity Log, then refreshes the animal table and the analytics. For an image, it shows the picture and provider.
- If a worker fails: the GUI shows an error message or “Image unavailable” and the application continues.

Because the long work is off the EDT, the window keeps repainting and accepting clicks during an import or an image download, so it never freezes.

7. Data Mining & Visualization Design

PawTrack uses the AnalyticsService class to analyze animal and adoption records stored in the SQLite database. It performs frequency analysis to identify the most adopted species and breeds, calculates average shelter stay durations, and aggregates adoption counts across the animal classification hierarchy. The results are organized into an AnalyticsReport and displayed through the application's analytics dashboard.

7.1 Data Mining and Aggregation

The application analyzes completed adoption records by matching each adoption record with its corresponding

animal. It groups and counts the records according to species and breed, calculates shelter stay durations, and performs recursive aggregation through the classification tree.

Table 7. Data Mining and Aggregation Operations		
Analysis	**Aggregation Method**	**Purpose**
Adoptions by Species	Counts completed adoptions for each species and sorts the results from highest to lowest frequency.	Identifies the species with the highest number of adoptions.
Adoptions by Breed	Counts completed adoptions for each recorded breed and ranks them by frequency.	Identifies the most adopted breeds.
Average Shelter Stay	Calculates the days between an animal's intake and adoption dates, then computes the overall and per-species averages.	Determines how long animals typically stay in the shelter before adoption.
Adoptions by Classification	Uses recursive traversal to count adopted animals under each classification node, including all its descendant nodes.	Provides adoption totals across the animal classification hierarchy.

The analysis reads the live animal and adoption records from the SQLite database each time it runs. The analytics are refreshed after interest is recorded, an animal is adopted, or a CSV file is imported, and whenever Refresh Analytics is clicked, so the results and charts change whenever the underlying data changes.

7.2 Recursive Classification Analysis

PawTrack uses the ClassificationService and ClassificationNode classes to organize animal species and breeds into a hierarchical tree structure. The hierarchy allows classifications to have multiple levels without a fixed depth.

For example:

Dog > Sporting Group > Retriever > Golden Retriever

The application uses recursive methods to search for classifications by name or full path, retrieve classification paths, and traverse the hierarchy. During analytics processing, the AnalyticsService uses recursive aggregation to calculate the total number of adopted animals within each classification, including its descendants. This allows staff to analyze adoption patterns at different levels of the classification hierarchy rather than relying only on individual species or breed names.

7.3 Data Visualization

PawTrack uses a bar chart, drawn by the BarChartPanel class, to show the aggregated results. A bar chart compares numbers across categories such as species, breeds, and classifications. The AnalyticsPanel provides a metric dropdown and a **Refresh Analytics** button.

Table 8. Analytics Dashboard Visualizations		
Chart	**Data Displayed**	**Purpose**
Adoptions by Species	Completed adoptions per species	Compare adoption frequency
Top Adopted Breeds (Top 8)	The eight breeds with the most adoptions	Highlight the most adopted breeds
Average Shelter Stay (days) by Species	Average shelter days per species	Compare stay length
Adoptions by Classification (recursive)	Adoptions per classification, including descendants	Show adoption spread across the hierarchy

The dashboard also shows a text summary: total animals, completed adoptions, most adopted species and breed, average shelter stay, and adopters on waitlists. Together, these results help staff spot adoption patterns and monitor shelter stay durations.